CENTRO UNIVERSITÁRIO FACIMP WYDEN — UNIFACIMP
CURSO DE BACHARELADO EM SISTEMAS DE INFORMAÇÃO
DISCIPLINA: DESENVOLVIMENTO WEB

GUILHERME CORREIA
[NOME DO INTEGRANTE 2]
[NOME DO INTEGRANTE 3]

AUTORENT: SISTEMA WEB PARA GESTÃO DE FROTA E CONTROLE DE LOCAÇÃO DE VEÍCULOS

Etapa 1 — Definição do Problema, Requisitos Iniciais e Planejamento Técnico

> Documento de concepção e planejamento técnico apresentado à disciplina de Desenvolvimento Web, do curso de Sistemas de Informação da UNIFACIMP Wyden, referente à entrega da Etapa 1 do Projeto Prático.
>
> **Professor:** Esp. Welliton Cunha.

Nome Provisório do Projeto	AutoRent — Gestão de Frota e Locação de Veículos
Curso e Turma	Sistemas de Informação — 2026.2 (Quarta-feira)
Repositório Git (GitHub)	https://github.com/gcorreia-devsec/autorent-web

IMPERATRIZ — MA
OUTUBRO DE 2026

1 PROBLEMÁTICA E CONTEXTO ATUAL

O gerenciamento do pátio e dos contratos de aluguel em pequenas e médias locadoras de veículos regionais ainda sofre bastante com a falta de ferramentas simples e centralizadas. O nosso projeto, chamado provisoriamente de **AutoRent**, surgiu ao observar a rotina dessas locadoras menores, onde o controle de quais carros estão livres, quais estão alugados com clientes e quais pararam na oficina para manutenção é feito de forma muito improvisada.

No contexto atual, a maioria desses estabelecimentos controla sua frota usando planilhas soltas no Excel, agendas de papel ou trocando mensagens em grupos de WhatsApp da equipe. Na prática, isso gera vários problemas: quando um cliente chega no balcão perguntando se determinado carro está disponível, o atendente perde tempo conferindo anotações ou ligando para o pátio. Além disso, sem uma trava no sistema, é comum acontecerem erros de duas pessoas reservarem o mesmo veículo para o mesmo horário, atrasos na cobrança de diárias e perda do histórico de quem alugou cada carro.

2 PÚBLICO-ALVO E PRINCIPAIS PESSOAS USUÁRIAS

O sistema foi pensado para o uso interno de pequenas e médias locadoras de veículos, atendendo três perfis principais de usuários:

- **Atendentes de Balcão:** São os operadores que mais usam o sistema na rotina diária. Precisam consultar rapidamente os veículos livres, cadastrar clientes, registrar a saída de um carro e dar baixa na devolução.
- **Administradores e Proprietários:** Precisam visualizar o quadro geral da frota, cadastrar novos veículos adquiridos, atualizar valores de diárias e consultar o histórico de contratos.
- **Responsáveis pelo Pátio e Manutenção:** Consultam a lista da frota para mudar o status de um veículo quando ele precisa de revisão mecânica ou lavagem, evitando que o balcão alugue um carro indisponível.

3 OBJETIVO DA SOLUÇÃO E ESCOPO INICIAL

O objetivo geral do **AutoRent** é desenvolver um sistema web integrado (frontend, API REST no backend e banco de dados relacional) para centralizar o cadastro da frota, dos clientes e o registro de retiradas e devoluções. Os principais benefícios esperados são a redução do tempo de atendimento no balcão, o fim das reservas duplicadas e a organização do histórico de locações.

Dentro do Escopo Inicial: Cadastro, listagem, edição e exclusão (CRUD) de veículos e clientes; registro de abertura e encerramento de locação; mudança automática do status do carro (Disponível, Alugado ou Manutenção) e tela inicial com filtro da frota.

Fora do Escopo Inicial: Pagamento online via cartão ou Pix; rastreamento veicular por GPS em

4 REQUISITOS INICIAIS DO SISTEMA

A partir do problema identificado e do escopo definido pela equipe, levantamos os requisitos funcionais e não funcionais iniciais que servirão de base para a estrutura do sistema e do banco de dados.

4.1 Requisitos Funcionais (RF)

Quadro 1 — Requisitos Funcionais iniciais do sistema AutoRent

Código	Funcionalidade	Descrição do Comportamento no Sistema
RF01	Gerenciar Veículos	Permitir cadastrar, listar, editar e excluir veículos da frota, registrando modelo, marca, placa, ano, valor da diária e status atual.
RF02	Gerenciar Clientes	Permitir cadastrar, consultar, atualizar e remover clientes, informando nome completo, CPF, número da CNH, telefone e e-mail.
RF03	Registrar Locação	Permitir abrir uma locação vinculando um cliente cadastrado a um veículo com status "Disponível", informando data de saída e previsão de entrega.
RF04	Atualizar Status	Alterar automaticamente o status do veículo para "Alugado" no momento da retirada e retornar para "Disponível" quando a devolução for concluída.
RF05	Filtrar Frota	Permitir que o usuário filtre a listagem de veículos pelo status atual (Disponível, Alugado ou em Manutenção).
RF06	Encerrar Locação	Permitir registrar a devolução do veículo, gravando a data de entrega e calculando o valor total conforme a quantidade de diárias.

Fonte: Elaborado pelos autores (2026).

4.2 Requisitos Não Funcionais (RNF)

- **RNF01 (Arquitetura da API):** O backend deve seguir o padrão REST utilizando JSON e dividindo as responsabilidades nas pastas *routes*, *controllers* e *models*.
- **RNF02 (Integridade Relacional):** O banco de dados PostgreSQL deve impedir duplicidade de placa, CPF e CNH (restrição *UNIQUE*) e validar os vínculos de locação por chaves estrangeiras.
- **RNF03 (Segurança de Configuração):** Dados de acesso ao banco (usuário, senha e porta) devem ficar isolados em arquivo `.env` fora do Git, e os comandos SQL devem usar parâmetros (`$1, $2`) contra *SQL Injection*.
- **RNF04 (Usabilidade):** A interface deve ser simples, responsiva e validar campos obrigatórios antes do envio das requisições.

5 TECNOLOGIAS ESCOLHIDAS E LISTA DE FERRAMENTAS

As tecnologias foram selecionadas de acordo com as ferramentas trabalhadas na disciplina de Desenvolvimento Web, utilizando JavaScript em todas as pontas do projeto para facilitar a colaboração do grupo.

Quadro 2 — Lista de tecnologias escolhidas e justificativa técnica

Camada / Área	Tecnologia Escolhida	Justificativa de Uso no Projeto
Frontend	HTML5, CSS3 e JavaScript (ES6+)	Construção das telas de cadastro e painéis da frota de forma leve, consumindo os dados da API com `fetch()`.
Backend (Runtime)	Node.js + npm	Execução de JavaScript no servidor com bom desempenho em requisições assíncronas e gerenciamento de pacotes via `npm`.
Framework Backend	Express.js	Estruturação rápida do servidor HTTP, separação das rotas (`express.Router`) e leitura de dados em JSON (`express.json`).
Banco de Dados	PostgreSQL + pgAdmin 4	SGBD relacional confiável e gratuito para armazenar as tabelas de veículos, clientes e contratos de locação.
Driver e Ambiente	Pacotes `pg`, `dotenv` e `nodemon`	O `pg` realiza a conexão via *Pool*; o `dotenv` carrega as variáveis do arquivo `.env` e o `nodemon` reinicia o servidor durante o desenvolvimento.
Prototipação	Figma	Ferramenta colaborativa escolhida para desenhar a proposta visual das telas e o fluxo de uso do sistema.
Versionamento e Testes	Git, GitHub, VS Code e Postman	Git e GitHub para controle de versão do código da equipe, VS Code como editor padrão e Postman para testar os endpoints da API.

Fonte: Elaborado pelos autores (2026).

6 RISCOS, DÚVIDAS E PREMISSAS IDENTIFICADOS

- **Riscos Identificados:** Possibilidade de conflitos de código (*merge conflicts*) no GitHub caso dois integrantes alterem o mesmo arquivo ao mesmo tempo, além de eventuais dificuldades técnicas ao validar conflitos de datas nas reservas de veículos.
- **Dúvidas Iniciais:** Confirmar com o professor se a execução final do projeto será avaliada em ambiente local (`localhost`) nos computadores da equipe ou se será necessário hospedar a aplicação e o banco de dados em nuvem.
- **Premissas:** Assume-se que todos os membros da equipe possuem as ferramentas básicas instaladas (Node.js, PostgreSQL, VS Code e Git) e disponibilidade para dividir as tarefas semanalmente.